

CENTRO UNIVERSITÁRIO UNIALFA
CURSO DE ENGENHARIA DE SOFTWARE

APLICAÇÃO WEB DE CONTROLE DE GASTOS PESSOAIS

Goiânia
2026

CENTRO UNIVERSITÁRIO UNIALFA
CURSO DE ENGENHARIA DE SOFTWARE

APLICAÇÃO WEB DE CONTROLE DE GASTOS PESSOAIS

Trabalho apresentado ao Centro Universitário UNIALFA, como requisito avaliativo da disciplina de Fundamentos da Engenharia de Software.

Discente: Henrique de Lima Souza
Docente: George Mendes Marra

Goiânia
2026

SUMÁRIO

1 INTRODUÇÃO

2 METODOLOGIA

3 TECNOLOGIAS UTILIZADAS

4 ESTRUTURA DO SISTEMA

4.1 Frontend (Interface do Usuário)

4.2 Backend (APIs no Next.js)

4.3 Banco de Dados (Prisma)

5 REQUISITOS DO SISTEMA

5.1 Requisitos Funcionais (RF)

5.2 Requisitos Não Funcionais (RNF)

6 FLUXO PRINCIPAL DO USUÁRIO

7 CONSIDERAÇÕES FINAIS

REFERÊNCIAS

1 INTRODUÇÃO

Este trabalho apresenta o desenvolvimento de uma aplicação web de controle de gastos pessoais. O sistema foi planejado para ajudar o usuário a organizar melhor sua vida financeira, permitindo o cadastro de receitas e despesas, a separação por categorias, a visualização de um dashboard e a geração de relatórios.

A proposta do projeto é oferecer uma solução prática e fácil de usar, reunindo em um único ambiente as principais funções para acompanhamento financeiro. Além disso, o sistema permite a exportação de dados em CSV e PDF, facilitando a análise das informações.

2 METODOLOGIA

O presente trabalho foi desenvolvido por meio de pesquisa bibliográfica, tendo como base a documentação oficial das tecnologias utilizadas, bem como obras de referência na área de Engenharia de Software. A pesquisa bibliográfica permitiu embasar as decisões técnicas do projeto, orientando a escolha das ferramentas e a estruturação da aplicação.

A partir do referencial teórico levantado, foi realizado o desenvolvimento prático da aplicação web, seguindo boas práticas de desenvolvimento de software, como separação de responsabilidades, componentização da interface, segurança no acesso às rotas e persistência adequada dos dados. O processo de desenvolvimento foi conduzido de forma iterativa, com validação dos requisitos funcionais e não funcionais definidos para o sistema.

3 TECNOLOGIAS UTILIZADAS

- **Next.js:** framework utilizado no frontend e no backend da aplicação.
- **TypeScript:** utilizado para trazer mais segurança e organização ao código.
- **Prisma:** ORM responsável pelo acesso e pela modelagem do banco de dados.
- **SQLite:** banco de dados utilizado para persistência local das informações.
- **Autenticação por sessão:** utilizada para login, sessão e proteção de rotas privadas.
- **Node.js e npm:** utilizados no gerenciamento do projeto e dos scripts.

4 ESTRUTURA DO SISTEMA

4.1 Frontend (Interface do Usuário)

A parte visual do sistema foi organizada com páginas e componentes reutilizáveis. Entre os principais elementos estão a tela inicial, cadastro, login, dashboard, categorias, transações, relatórios e tela de usuário. Também foram utilizados componentes de interface como botões, cards, inputs, selects, tabelas e gráficos.

4.2 Backend (APIs no Next.js)

O backend foi construído com rotas de API dentro do próprio Next.js. Essas rotas são responsáveis pela autenticação do usuário, gerenciamento de categorias, cadastro e controle de transações, geração de relatórios e exportações em CSV e PDF.

4.3 Banco de Dados (Prisma)

O banco de dados é acessado por meio do Prisma. No schema são definidos os principais modelos da aplicação, como usuário, sessão, categoria e transação. Também foi previsto um arquivo de seed para popular o banco com dados iniciais, como categorias padrão e usuário de demonstração.

5 REQUISITOS DO SISTEMA

5.1 Requisitos Funcionais (RF)

RF01: O sistema deve permitir que o usuário realize cadastro e login com segurança.

RF02: O sistema deve permitir o registro de transações financeiras, incluindo despesas e receitas.

RF03: O sistema deve permitir informar, nas transações, dados como tipo, valor, data, descrição e categoria.

RF04: O sistema deve permitir criar, listar, editar e excluir categorias.

RF05: O sistema deve permitir criar, listar e filtrar transações.

RF06: O sistema deve apresentar um dashboard com visão geral das finanças do usuário.

RF07: O sistema deve permitir gerar relatórios por período ou categoria.

RF08: O sistema deve permitir gerar gráficos de barras e gráfico de pizza para melhor controle do usuário.

RF09: O sistema deve permitir exportar os dados em formato CSV.

RF10: O sistema deve permitir exportar relatórios em formato PDF.

5.2 Requisitos Não Funcionais (RNF)

RNF01: O sistema deve possuir autenticação segura para acesso às áreas restritas.

RNF02: O sistema deve proteger rotas privadas, como o dashboard, permitindo acesso apenas a usuários autenticados.

RNF03: O sistema deve ser desenvolvido com TypeScript para aumentar a segurança e a organização do código.

RNF04: O sistema deve utilizar Prisma para garantir melhor estrutura no acesso ao banco de dados.

RNF05: O sistema deve possuir interface organizada com componentes reutilizáveis.

RNF06: O sistema deve disponibilizar exportação compatível com ferramentas como Excel e planilhas.

RNF07: O sistema deve manter os dados persistidos de forma adequada no banco de dados.

RNF08: O sistema deve apresentar navegação simples e prática para o usuário.

6 FLUXO PRINCIPAL DO USUÁRIO

1. O usuário cria uma conta ou faz login no sistema.
2. Após a autenticação, o usuário acessa o dashboard.
3. O usuário pode visualizar categorias existentes e também criar, editar ou remover categorias.
4. Na área de transações, o usuário registra informações como tipo, valor, data, categoria e descrição.
5. Por fim, o usuário pode gerar relatórios e exportar os dados em CSV ou PDF.

7 CONSIDERAÇÕES FINAIS

Este projeto foi importante para demonstrar como a tecnologia pode ajudar no controle financeiro pessoal. Com as funções de cadastro, login, registro de

transações, categorias, dashboard e relatórios, o sistema atende ao seu objetivo. Dessa forma, o trabalho apresenta uma solução útil, prática e adequada para o dia a dia do usuário.

REFERÊNCIAS

PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2018.

VERCEL. *Next.js Documentation*. Disponível em: <<https://nextjs.org/docs>>. Acesso em: jun. 2026.

PRISMA. *Prisma Documentation*. Disponível em: <<https://www.prisma.io/docs>>. Acesso em: jun. 2026.

MICROSOFT. *TypeScript Documentation*. Disponível em: <<https://www.typescriptlang.org/docs>>. Acesso em: jun. 2026.

NODE.JS. *Node.js Documentation*. Disponível em: <<https://nodejs.org/docs>>. Acesso em: jun. 2026.